Performance Analysis of Apache ZooKeeper's Write Pipeline

Software Performance Engineering — Project Stage II

Gil Jorge (110062), João Palma (109242), and Gabriel Petti Tavares (120691)

Instituto Superior Técnico, Universidade de Lisboa — Group 18
gil.jorge@tecnico.ulisboa.pt, joaofontepalma@tecnico.ulisboa.pt,
gabriel.p.tavares@tecnico.ulisboa.pt

Abstract. ZooKeeper answers reads at any server but sends every write through one leader and a quorum. We study the cost of that write pipeline on Google Compute Engine with a 2^{6-1} factorial experiment (two replicates), one replicated Universal Scalability Law curve and five eBPF probes. The write ratio explains 60% of the variation of throughput and the number of clients 94% of the variation of the write tail latency. Turning `fsync` off helps at light load (+39%) and hurts at saturation (−17%). The curve needs only a contention term ($\sigma = 0.039$); the disk is not where that contention comes from, but our data do not single out one stage of the leader either. We also report a client-side logging artifact that had distorted the Stage I fits.

Git Commit Identifier: `9a52eabcfe0019b8ab4ac96fb7bb2195694d70a0`

1 Introduction

Apache ZooKeeper is a coordination service used by Kafka, HBase and Hadoop for leader election, configuration and locks [1, 2]. Its replication protocol, Zab [3], makes reads and writes behave very differently: a read is answered by the server the client is connected to, while a write goes to the leader, is logged and acknowledged by a majority and only then committed. Reads should therefore scale with the number of servers and writes should not, and the write pipeline is where we look for the limits of scalability.

Stage I measured one laptop with Docker and fitted the Universal Scalability Law (USL) [4, 5]. In Stage II we (1) found and removed a client-side artifact in the Stage I data (Section 3); (2) ran a fractional factorial experiment on dedicated virtual machines to rank the factors that drive throughput and write tail latency (Sections 5–6.1); (3) fitted one replicated USL curve and recorded ZooKeeper's per-stage timers at every load level (Section 6.2); and (4) observed the leader with eBPF (Section 6.3).

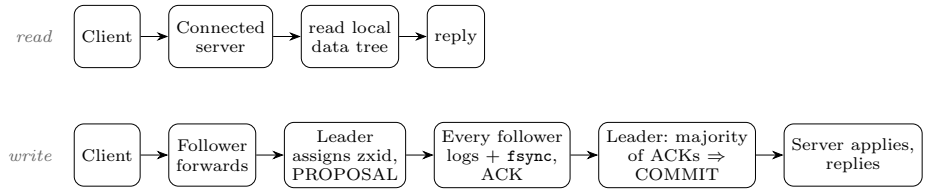


Fig. 1. Read and write paths in ZooKeeper.

2 System Description

ZooKeeper keeps a tree of small data nodes (znodes) in memory and persists every change in a transaction log. $N = 2f + 1$ servers elect a *leader*, tolerate f failures and stop accepting writes when no majority is reachable [3]. Writes are linearizable; reads are sequentially consistent per session and may be slightly stale, which is what allows a follower to answer them alone [1]. Figure 1 shows both paths.

Inside the leader a write passes a chain of processors. Request preparation (one thread) validates it and assigns the zxid; the log-sync stage (one thread) appends it to the log and calls `fsync`, writing every request that is queued at that moment in one batch (group commit); the commit stage applies it once a majority has acknowledged. The followers run the same log-sync stage before they acknowledge. A read costs one hop and no coordination; a write costs a forward, a proposal to every follower, a log write and `fsync` on each, acknowledgments and a commit.

3 Stage I

In Stage I the same configuration (3 servers, 20 clients, 30% writes) gave 6,364 ops/s in the sweep and 14,832 ops/s when we repeated it later. The jar used for the sweep bundled Logback, whose default configuration logs the ZooKeeper client at DEBUG level: it printed about 145,000 lines in a 15 s run, one per operation, and all client threads waited on the console. The fix is a one-line Logback configuration, and the runner now aborts any run in which the client prints more than a few lines. This changes the Stage I conclusion. The fits had given the same contention coefficient (≈ 0.23) for 1, 3 and 5 servers, which we read as server-side contention unrelated to Zab; a bottleneck in the client also produces a coefficient that does not depend on the number of servers. With the quiet client the coefficient was 0.025, 0.088 and 0.112 for 1, 3 and 5 servers. In the same comparison, repetitions of one variant on the laptop differed by up to a factor of two, and the sweep had one run per point in increasing order of load. Stage II therefore uses separate machines for client and servers, replicates and a randomized order. The Stage I experiments that we did not repeat on the new testbed (YCSB, leader failover, injected network delay) are summarized in Appendix A and are not used for any conclusion of this report.

4 Testbed and Methodology

Deployment. All Stage II runs use Google Compute Engine in one zone (europe-west1-b). The load generator has its own n2-standard-8 machine (8 vCPUs, 32 GB); every ZooKeeper server has an n2-standard-2 (2 vCPUs, 8 GB). All run Debian 12 (kernel 6.1) and OpenJDK 17. ZooKeeper 3.9.3 runs natively with a 1 GB heap. Each server has two extra 20 GB disks, pd-balanced and pd-ssd, formatted with ext4; the transaction log goes on one of them according to the factor *log disk*, and snapshots stay on the boot disk. Before each block of runs the servers are stopped, their data is erased and they are restarted with the new configuration; the runner then checks on every server that the `forceSync` flag and the log directory of the running process are the intended ones.

Load generator. A Java client opens one ZooKeeper session per simulated client, spread over all servers by the connect string. Each client is a closed loop of synchronous calls without think time: it picks one of 200 znodes uniformly and sends a `setData` with probability equal to the write ratio, otherwise a `getData`. All sessions connect before a barrier releases them. Operations that start in the first 5 s are executed but not recorded, so that the JIT compilation of client and server code and the filling of caches fall outside the measurement; we did not measure the length of that transient separately, but every series starts with a discarded run as well. A factorial run then measures 20 s and a USL run 30 s. The slowest factorial run still completed more than 30,000 writes, so its 99th percentile rests on over 300 samples. Percentiles are exact (all latencies are kept and sorted). The client machine logs its own CPU every second; it never exceeded 47% busy, so the client was not the bottleneck.

ZooKeeper's per-stage timers. The `mntr` command reports, per stage, a count and a sum: the time a request waits for the request-preparation thread, the time in the log-sync queue and flush, the `fsync` time, requests per `fsync` and the time from proposal to quorum acknowledgment. We take a snapshot of every server before and after each run; the difference of the sums over the difference of the counts is the mean of that run. The timers have 1 ms resolution and truncate, so the `fsync` is also measured with eBPF.

Threats to validity. The virtual machines share physical hosts with other tenants. Replicates and the random order spread that noise over all treatments but do not remove it: two replicates of the same factorial run differ by 11% at the median and by up to 48%. One zone means short round trips between servers. `forceSync=no` is unsafe in production and serves only as a probe.

5 Experimental Design

Parameters. Table 1 lists the fixed system and workload parameters; the six factors and their levels are in Table 2.

Table 1. Fixed parameters. Defaults are those of ZooKeeper 3.9 [6].

Parameter	Value
Observers; `leaderServes`	0; yes (default)
Snapshot device (`dataDir`)	boot disk (pd-standard)
`snapCount`, `preAllocSize`	100,000, 64 MB (defaults)
`maxBatchSize`, `flushDelay`	1000, 0 (defaults)
`globalOutstandingLimit`, `maxClientCnxns`	1000 (default), no limit
`tickTime`, `initLimit`, `syncLimit`	2000 ms, 10, 5
Server JVM; versions	JDK 17, 1 GB heap, G1; ZooKeeper 3.9.3, client 3.9.0
Operations; key space	`getData/setData` only; 200 znodes, uniform, new pool per run
Client model; session placement	closed loop, synchronous, no think time; random server
Run length	5 s warm-up + 20 s (factorial) or 30 s (USL) measured

Table 2. Factors and levels. "Hard": changing it needs a rebuild of the servers.

Factor	−1	+1	Hard	Why
A number of servers	3	5	yes	quorum size: proposals and ACKs per write; the two usual production sizes
B write ratio	0.1	0.5		share of requests that take the write pipeline (reads stay local)
C clients	10	160		light load vs well into saturation
D forceSync	yes	no	yes	removes the fsync from the write path: tests the log-sync stage
E payload (bytes)	128	4096		bytes logged, sent to every follower and synced per write
F log disk	pd-balanced	pd-ssd	yes	speed of the device under the fsync (dedicated log disk)

Metrics. The primary response is throughput (successful operations per second), the quantity the USL models. The secondary response is the 99th-percentile write latency, because a coordination service sits on the critical path of other systems. We do not use the mean latency as a separate response: in a closed loop without think time Little's law gives the mean response time $R = N/X$, so it carries no information beyond throughput. Failed operations (there were none) and the client's CPU are recorded as validity checks.

Factors and levels. Each factor tests one guess about where a write spends its time (Table 2). We chose 10 and 160 clients from the Stage I curve, where 10 was below the knee. On the new testbed the throughput per client at 10 clients is already about 70% of that of a single client (Section 6.2), so the low level is light load and the high level is deep in saturation. A write ratio of 0.1 is close to the 10:1 read/write ratio cited in [7]; 0.5 loads the write path. Three and five servers are the usual sizes. The payloads bracket small metadata and configuration blobs. Persistent Disk performance depends on disk and machine size, so the two disk types are levels of this testbed only.

Design. We use the half fraction 2^{6-1} with $F = ABCDE$ (defining relation $I = ABCDEF$) [8]. It has resolution VI: main effects are aliased with five-

Table 3. Sign table of the 2^{6-1} design with the mean throughput of the two replicates ($\bar{y}$, ops/s).

Run	A	B	C	D	E	F	$\bar{y}$	Run	A	B	C	D	E	F	$\bar{y}$
1	−	−	−	−	−	−	25667	17	−	−	−	−	+	+	16841
2	+	−	−	−	−	+	25289	18	+	−	−	−	+	−	16902
3	−	+	−	−	−	+	8804	19	−	+	−	−	+	−	5664
4	+	+	−	−	−	−	9702	20	+	+	−	−	+	+	6245
5	−	−	+	−	−	+	53235	21	−	−	+	−	+	−	45392
6	+	−	+	−	−	−	79861	22	+	−	+	−	+	+	54084
7	−	+	+	−	−	−	19988	23	−	+	+	−	+	+	15242
8	+	+	+	−	−	+	20315	24	+	+	+	−	+	−	16408
9	−	−	−	+	−	+	29633	25	−	−	−	+	+	−	22213
10	+	−	−	+	−	−	32421	26	+	−	−	+	+	+	26116
11	−	+	−	+	−	−	12408	27	−	+	−	+	+	+	9549
12	+	+	−	+	−	+	11947	28	+	+	−	+	+	−	9646
13	−	−	+	+	−	−	49811	29	−	−	+	+	+	+	38703
14	+	−	+	+	−	+	60824	30	+	−	+	+	+	−	39773
15	−	+	+	+	−	+	12908	31	−	+	+	+	+	−	12076
16	+	+	+	+	−	−	23112	32	+	+	+	+	+	+	15022

factor interactions and two-factor interactions with four-factor ones, so all six main effects and fifteen two-factor interactions are estimated from 32 runs per replicate if interactions of order four and higher are negligible. A quarter fraction would have resolution IV and alias two-factor interactions in pairs, which is where hypotheses H2–H4 sit. Table 3 is the sign table: the full 2^5 table of A–E with F as the product of the other five columns. With two replicates there are 64 runs and 32 degrees of freedom for error.

Restricted randomization. A, D and F need a rebuild of the servers. Runs sharing their levels form a block (eight blocks of four runs per replicate, 16 rebuilds in total). The block order is randomized in each replicate, and so is the run order inside each block. The plan was generated with a fixed seed before the first run. This is a split-plot structure: the block, not the run, is the experimental unit of A, D and F. Our main analysis tests all effects against the pooled run error; for A, D and F we repeat the test against a whole-plot error estimated from the 16 block means (replicate × treatment, 7 degrees of freedom).

Analysis. With s_{ij} the sign of run i for effect j, $\bar{y}_i$ the mean of the $r = 2$ replicates and $n = 32$,

$$q_j = \frac{1}{n}\sum_i s_{ij}\,\bar{y}_i, \quad SS_j = nrq_j^2, \tag{1}$$

$$s_e^2 = \frac{\sum_{i,k}(y_{ik} - \bar{y}_i)^2}{n(r-1)}, \quad q_j \pm t_{[0.975;\,n(r-1)]}\frac{s_e}{\sqrt{nr}}, \tag{2}$$

and effect j explains $100\,SS_j/SST$ percent of the variation. Testing 31 effects at 95% would produce about two false positives, so we rank effects by their share of the variation. When $y_{\max}/y_{\min} > 10$ we analyse $\log_{10} y$; residual plots are in Appendix C. We tested the analysis code on synthetic data with known effects before using it.

Table 4. Largest effects ($\log_{10}$ model): q, the factor (×) from the low to the high level, the 95% interval of q and the share of the variation (%). †: interval includes zero.

	throughput (ops/s), $\log_{10}$			write p99 (ms), $\log_{10}$		
Effect	q (×)	95% CI	%	q (×)	95% CI	%
C	0.149 (×1.98)	[0.133, 0.164]	25.6	0.534 (×11.68)	[0.511, 0.556]	93.7
B	-0.228 (×0.35)	[-0.244, -0.213]	60.3	0.041 (×1.21)	[0.018, 0.064]	0.6
E	-0.066 (×0.74)	[-0.081, -0.051]	5.0	0.061 (×1.32)	[0.038, 0.084]	1.2
C×D	-0.056 (×0.77)	[-0.071, -0.041]	3.6	0.059 (×1.31)	[0.036, 0.081]	1.1
A	0.029 (×1.14)	[0.014, 0.044]	1.0	0.021 (×1.10)†	[-0.001, 0.044]	0.2
D	0.015 (×1.07)†	[-0.001, 0.030]	0.3	-0.042 (×0.82)	[-0.065, -0.019]	0.6
E×F	0.021 (×1.10)	[0.006, 0.036]	0.5	-0.004 (×0.98)†	[-0.027, 0.018]	0.0
A×B	0.000 (×1.00)†	[-0.016, 0.015]	0.0	-0.034 (×0.85)	[-0.057, -0.012]	0.4
A×C	0.018 (×1.09)	[0.003, 0.033]	0.4	0.010 (×1.05)†	[-0.013, 0.033]	0.0
B×C	-0.016 (×0.93)	[-0.032, -0.001]	0.3	-0.020 (×0.91)†	[-0.043, 0.003]	0.1
B×E	0.003 (×1.01)†	[-0.012, 0.019]	0.0	-0.022 (×0.90)†	[-0.045, 0.000]	0.2

Hypotheses. Written before the runs: **H1**: turning `fsync` off (D) raises throughput and lowers the write p99. **H2**: that benefit grows with the write ratio (B×D), since reads never touch the log. **H3**: a faster log disk (F) helps only while `fsync` is on (D×F). **H4**: five servers (A) lower throughput and raise the write p99, mostly through writes (A×B). **H5**: the number of clients (C) has the largest main effect on throughput.

6 Results

6.1 Factorial experiment

All 64 runs completed without errors. Throughput ranged from 5,335 to 81,088 ops/s and the write p99 from 1.2 to 61 ms. Both ratios exceed 10, and the residuals of the additive model widen with the predicted value, so both responses are analysed on $\log_{10}$, where the residuals look homogeneous and roughly normal (Fig. 5). Effects are then multiplicative factors (Table 4). The error explains 2.1% (throughput) and 1.3% (p99) of the variation. That is a share of the total, and it is small because the factors change throughput up to fifteen-fold; in absolute terms the replicates still differ by about 11%.

Throughput. The write ratio (B) has the largest effect: going from 10% to 50% writes divides throughput by 2.9 (60% of the variation). If a read costs the servers r and a write w, a throughput ratio q gives $(0.9r+0.1w)/(0.5r+0.5w) = q$. That derivation assumes throughput is set by server capacity, which is only true at 160 clients; there $q = 0.32$ and $w \approx 12r$, and at 10 clients $w \approx 8r$. A write therefore costs roughly ten reads. The number of clients (C) comes second (×1.98, 26%), and 4 KiB payloads (E) cost 26% of throughput (5%).

The largest interaction is clients × `forceSync` (C×D, 3.6%, Fig. 2). With 10 clients, turning `fsync` off raises throughput from 12,381 to 17,150 ops/s (geometric means, +39%); with 160 clients it lowers it from 31,776 to 26,270 ops/s (−17%). The reversal holds in every cell: at 160 clients the ratio without and

with `fsync` is between 0.79 and 0.87 at all levels of the other factors. At light load each write waits for its own `fsync`, so removing it shortens every request. At saturation group commit already spreads one `fsync` over many requests (Section 6.2). Our explanation for the loss is that without `fsync` the log-sync stage flushes almost every request on its own (1.9 per flush against 16–17 with `fsync`, Table 8), and the extra flushes and acknowledgment rounds cost the leader CPU it no longer has. We did not test this directly.

Five servers raise throughput by 14% (×1.14, 1% of the variation). The effect is significant against both the pooled and the whole-plot error (interval ×1.08–1.21 in the whole-plot test), and it depends on the load (A×C): ×1.05 at 10 clients and ×1.24 at 160. It is the same at both write ratios (A×B = ×1.00), so it cannot come from the extra servers answering more reads. A better fit is the leader's share of the sessions. Sessions are spread over all servers, so the leader serves a third of them with three servers and a fifth with five, and the probe runs show that client connections take 0.4 cores of the leader's two (Section 6.3). With five servers the leader has more CPU left for coordination, which matters only when it is saturated. The log disk (F) has no main effect under either error and appears only in a small E×F interaction (0.5%).

Write tail latency. Load decides the write p99: 160 clients multiply it by 11.7 (94% of the variation), from 2.7 to 31.7 ms (geometric means). This is queueing: with sixteen times more clients and less than twice the throughput, each request finds a longer queue at the leader. The other effects change it by at most a third: payload (×1.32), C×D (without `fsync` the p99 at 10 clients falls from 3.4 to 2.2 ms and at 160 clients stays at 31–33 ms), write ratio (×1.21), `forceSync` (×0.82) and five servers (×1.10, significant only in the whole-plot test).

Hypotheses. **H1** holds only at light load. The main effect of D on throughput (×1.07) is not significant against the pooled error and is significant against the whole-plot error; either way it is dwarfed by C×D, which changes its sign. D lowers the write p99 by 18%. **H2** is rejected (B×D, ×1.04, not significant). **H3** is rejected: neither F nor D×F is detectable. **H4** is rejected for throughput (five servers are faster) and holds in part for the p99, which five servers raise with 10% writes (7.4 to 9.6 ms) and not with 50% (A×B, ×0.85). **H5** is rejected for throughput, where the write ratio matters more, and holds for the write p99.

6.2 USL curve

We measured one configuration (3 servers, 30% writes, 128 B) at 13 client counts from 1 to 160, with three replicates per level in random order. In the notation of the course,

$$X(N) = \frac{\lambda N}{1 + \sigma(N-1) + \kappa N(N-1)}, \tag{3}$$

where λ is the throughput of one client, σ the contention (serial) coefficient and κ the crosstalk (coherency) coefficient. The fit minimizes relative error, so

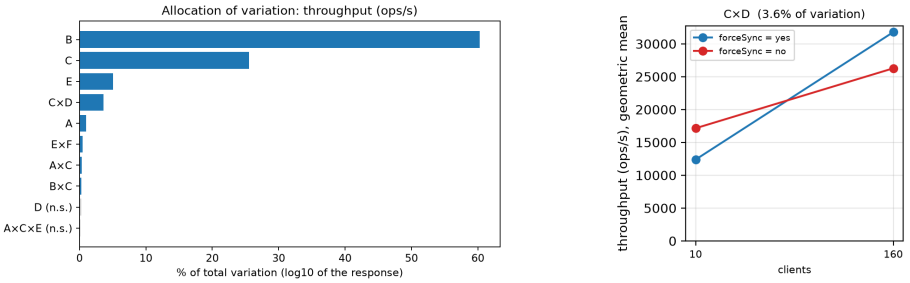


Fig. 2. Left: share of the variation of $\log_{10}$ throughput per effect (ten largest). Right: clients × `forceSync`.

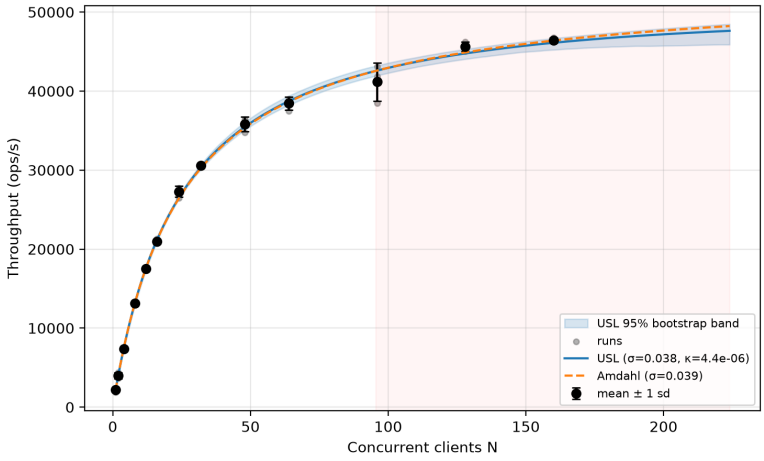


Fig. 3. Throughput against clients: runs, mean ± one standard deviation, the USL fit with its 95% bootstrap band and the Amdahl model. The shaded region was held out for validation.

that 3% counts the same at 600 and at 6,000 ops/s. The intervals come from a bootstrap over the replicates. We compare the full model with the contention-only (Amdahl) model by AIC and an F-test, and validate it by fitting $N \leq 64$ and predicting the three highest loads. The number of servers is not varied here; its effect is factor A above.

Throughput rises from 2,165 ops/s with one client to 46,436 ops/s with 160 (Fig. 3); the client stayed below 28% CPU. The fit gives $\lambda = 2{,}082$ ops/s (95% interval 1,996–2,169) and $\sigma = 0.0385$ (0.0345–0.0414). The crosstalk term is not supported: the Amdahl model has the lower AIC (-241.4 against -239.5), the F-test gives $p = 0.72$, and 37% of the bootstrap fits put κ at zero. We therefore use the Amdahl fit ($\lambda = 2{,}088$, $\sigma = 0.039$) and claim no retrograde region within 160 clients. It bounds throughput by $\lambda/\sigma \approx 53{,}500$ ops/s; at 160 clients the system

Table 5. Mean response time from the Amdahl fit and Little's law, against measured latencies (all operations).

Clients N	1	8	32	64	128	160
$R(N)$ predicted (ms)	0.48	0.61	1.06	1.66	2.85	3.45
N/X measured (ms)	0.46	0.61	1.05	1.66	2.81	3.45
median (ms)	0.22	0.28	0.49	0.87	1.64	2.42
95th perc. (ms)	1.15	1.79	2.95	4.48	7.50	8.75

Table 6. Leader per-stage times (ms, mean of the replicates) and requests per `fsync`. 1 ms timer resolution.

Leader, mean per run	$N = 1$	$N = 16$	$N = 64$	$N = 160$
prep queue (zxid) (ms)	0.02	0.05	0.35	1.37
sync queue (ms)	0.03	0.43	0.66	0.76
sync queue + flush (ms)	0.55	1.08	1.86	2.14
fsync (ms)	0.06	0.22	0.69	0.75
until quorum ACK (ms)	0.72	1.34	2.26	2.79
server-side write latency (ms)	0.87	1.61	3.26	5.57
requests per fsync	1.0	4.1	13.0	17.6

delivers 87% of that. The efficiency $X(N)/(N\lambda)$ falls to 0.74 at 10 clients and 0.13 at 160. Fitted on $N \leq 64$ only, the model predicts the throughput at 96, 128 and 160 clients within 1.9–5.1%.

Response time. Combining the Amdahl fit with Little's law for a closed loop without think time gives

$$R(N) = \frac{N}{X(N)} = \frac{1 + \sigma(N-1)}{\lambda}, \tag{4}$$

a floor of $1/\lambda = 0.48\,\text{ms}$ plus $\sigma/\lambda = 19\,\mu\text{s}$ for every added client: once the system saturates, each new client only adds waiting time. Table 5 compares it with the measured mean N/X and with the median and 95th percentile recorded by the client. The model and N/X agree within 3.7% at every level, which mostly restates the quality of the throughput fit. The mean lies between the measured median and 95th percentile at every level, as expected for a right-skewed distribution.

Per-stage times. Table 6 gives ZooKeeper's timers on the leader at four of the 13 levels (all levels: Fig. 4). From 1 to 160 clients the server-side write latency grows from 0.87 to 5.57 ms. The intervals overlap and do not add up: the time to quorum acknowledgment starts when the proposal is created and includes the leader's own log sync and the followers' round trip. In absolute terms the time to quorum acknowledgment grows most (+2.07 ms). The queue in front of request preparation grows most in relative terms, from 0.02 to 1.37 ms, and most of that growth happens beyond 64 clients. The log-sync queue, in contrast, levels off at about 0.7 ms, because the batch grows from 1 to 17.6 requests per `fsync`.

Table 7. Kernel view of the leader. Percentiles are upper bounds of $\log_2$ histogram buckets.

Configuration	fsync calls	fsync mean (μs)	fsync p99 (μs)	log disk p99 (μs)	run queue p99 (μs)	TCP retr.
fsync off, pd-balanced	0	–	–	16384	511	0
fsync off, pd-ssd	0	–	–	16384	511	0
fsync on, pd-balanced	21393	1064	2048	1024	511	0
fsync on, pd-ssd	23120	986	2048	512	511	0

Table 8. The probed runs seen by the client and by ZooKeeper (top) and the leader's CPU per thread, in cores (bottom).

Configuration	ops/s	no probes	overhead (%)	req./fsync	ZK fsync (ms)	to quorum ACK (ms)
fsync off, pd-balanced	29276	28795	-1.7	1.9	–	1.14
fsync off, pd-ssd	28739	24992	-15.0	1.8	–	1.16
fsync on, pd-balanced	26622	28731	7.3	17.4	0.77	2.56
fsync on, pd-ssd	26674	27301	2.3	16.0	0.70	2.52

CPU cores used by	fsync off, pd-balanced	fsync off, pd-ssd	fsync on, pd-balanced	fsync on, pd-ssd
replication to followers	0.67	0.67	0.55	0.59
client connections (I/O)	0.40	0.38	0.40	0.39
PrepRequestProcessor (zxid)	0.20	0.21	0.25	0.25
CommitProcessor	0.19	0.18	0.17	0.16
SyncRequestProcessor (log, fsync)	0.10	0.09	0.10	0.11
commit workers (apply, reply)	0.06	0.05	0.06	0.06
JIT compiler	0.05	0.06	0.03	0.03
whole JVM	1.73	1.70	1.61	1.62

6.3 eBPF probes

Four probes are our own bpftrace programs [9] and one is bcc's `runqlat`:

- `fsync` latency: syscall tracepoints of `fsync`/`fdatasync`, ZooKeeper's JVM only;
- block I/O latency per disk: `block_rq_issue` to `block_rq_complete` (bcc's `biolatency` uses a kprobe that is not traceable on kernel 6.1);
- run-queue latency of the JVM's threads (`runqlat`);
- TCP retransmissions and resets (`tcp` tracepoints);
- CPU per thread: a 99 Hz `profile` probe keyed by thread name. ZooKeeper names the thread of each stage and the JVM passes the names to Linux, so this gives the CPU of each stage.

They ran on the leader in separate runs (3 servers, 80 clients, 50% writes) at both levels of `forceSync` and log disk. Each probed run is preceded by a discarded warm-up run and placed between two identical runs without probes. Tables 7 and 8 give the results.

The `fsync` and the disk. With `forceSync=no` the probe sees no `fsync`, which confirms that factor D was applied. With it, an `fsync` takes 1.0 ms on average on both disks. Its histogram has two modes: 21–27% of the calls take 256–512 μs and 68–75% take 1–2 ms. The block-I/O histograms of the two disks are almost the same, with 85–88% of the requests in the 256–512 μs bucket. The p99 of pd-ssd falls one bucket lower only because slightly fewer requests cross 512 μs, so

Table 9. USL terms and number of servers: candidate stages, evidence and mitigations.

Term	Candidate stages	Evidence	Mitigation
σ	Work the leader does for every write: ordering, replication, log sync; its CPU	$\sigma = 0.039$; no thread near a full core; JVM at 1.6–1.7 of 2 cores; D and F do not raise throughput at saturation	More CPU for the leader; `leaderServes=no`; batched writes (multi-operations)
κ	Interference growing with pairs of clients (per-session state, GC, commit waits on reads)	Not supported ($p = 0.72$; 37% of bootstrap fits at zero)	—
λ	Cost of one request alone: round trips, serialization, payload, one `fsync`	$\lambda = 2{,}082$ ops/s (0.48 ms per request); 4 KiB payloads $\times 0.74$; quorum ACK 0.72 ms at $N = 1$	Asynchronous clients; smaller payloads
Servers (A)	Proposals to more followers vs. fewer sessions on the leader	$\times 1.14$, mostly at 160 clients; same at both write ratios; no retransmissions	Observers for reads; `leaderServes=no`

we cannot call one disk faster than the other, and that agrees with F having no effect. There are about 2.7 block requests per `fsync`. Our reading is that an ext4 `fsync` that must also commit the journal waits for several device round trips in sequence (data, journal, cache flush), which would explain both the 1–2 ms mode and why an `fsync` costs more than the device's p99. We did not trace the journal to confirm it. Without `fsync` the log disk sees only about 390 requests in 45 s, most of them taking 1–8 ms. These are background write-backs of dirty pages, merged into large requests, and no request waits for them.

CPU, scheduler and network. The leader's JVM uses 1.6–1.7 of its two vCPUs. The largest shares are replication to the followers (0.55–0.67 cores) and client connections (0.4); request preparation uses 0.20–0.25 and the log-sync thread 0.1. The run-queue p99 is below 0.5 ms and there are no retransmissions.

What the probe runs cannot show. The two runs without probes around each probed run differed by 6% to 43%, and the apparent probe overhead ranges from −15% to +7%. The spread in these runs is larger than any overhead we could detect, so we cannot quantify the overhead below about 15%. It is also larger than the difference between the configurations: with probes, the runs without `fsync` are 9% faster, while the unprobed runs show them 4% slower. We therefore draw no conclusion on throughput at 80 clients from these runs. We use them only for quantities that the probes measure directly (call counts, histograms, CPU shares) and that changed little between runs.

7 Discussion: What Limits Scalability

Table 9 maps each term of the USL, and the number of servers, to the parts of the pipeline it can come from, and lists the evidence we have and what could be done about it.

Contention (σ). The curve has one dominant limit, contention, and the disk does not cause it. An `fsync` costs about 1 ms on either disk, group commit spreads it over up to 17 requests at high load, and neither D nor F raises throughput at saturation. Where the contention does come from is less clear. In the probed runs no single thread is near a full core: the request-preparation thread uses a quarter of one and the log-sync thread a tenth. So it is not one serial stage running out of CPU. What is close to its limit is the leader's CPU as a whole, at 1.6–1.7 of two cores, and most of it goes to replication and client connections. Run-queue waits stay short, so the leader is not starved of CPU on each wake-up; but every write adds work to the same two cores, and the prep queue, which grows fastest at high load, is where requests wait for that work to drain. Two caveats apply. The probes measured a different configuration from the USL curve (80 clients and 50% writes against 30% writes), and the stage timers overlap. We read σ as the saturation of the leader's shared resources, mainly its CPU, under the per-write work of Zab. Three results support this reading: five servers help most at 160 clients, where they take sessions off the leader; turning `fsync` off hurts only at saturation; and the efficiency falls steadily from the lowest loads. A direct test would repeat the sweep with a larger leader machine or with `leaderServes=no`; we did not run it.

Mitigations. If σ is the leader's CPU, the useful changes relieve the leader: giving it more CPU, configuring it not to accept clients (`leaderServes=no`) [6], adding observers so reads do not need more voting servers, or batching writes in multi-operations so that the per-request replication work is spread the way group commit spreads the `fsync`. A faster log disk would not help, and turning `fsync` off costs throughput at high load as well as durability.

8 Conclusion

The workload decides ZooKeeper's throughput more than the configuration does: the share of writes explains 60% of its variation and the load 26%, while the log disk explains nothing and `fsync` matters only through its interaction with load. For the write tail latency the load explains 94%. One replicated curve is described by contention alone ($\sigma = 0.039$, $\lambda = 2{,}082$ ops/s), levels off towards about 53,500 ops/s and is predicted within 5% from the lower half of the loads. The `fsync` costs about 1 ms on both disks and group commit spreads it over up to 17 requests, so the disk does not limit scalability; our measurements point to the leader's CPU but do not prove it. The Stage I artifact also taught us to validate the load generator before interpreting its numbers.

Limitations. One zone; virtual machines with noisy neighbours (replicates differ by about 11%); a 2-vCPU leader, which makes CPU an early limit; disk results tied to 20 GB disks on small machines; one USL configuration; probes in a different configuration from the curve, with run-to-run noise larger than the probe overhead; a workload without creates, deletes, watches or multi-operations. The next experiments are the USL sweep with a larger leader and with `leaderServes=no`, and the same sweep with observers.

References

1. P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. ZooKeeper: Wait-free coordination for Internet-scale systems. In *USENIX Annual Technical Conference*, 2010.
2. Apache Software Foundation. ZooKeeper: A Distributed Coordination Service for Distributed Applications. ZooKeeper 3.9.6 documentation, 2024. `https://zookeeper.apache.org/doc/r3.9.6/zookeeperOver.html`.
3. F. P. Junqueira, B. C. Reed, and M. Serafini. Zab: High-performance broadcast for primary-backup systems. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2011.
4. Distributed Systems Conference. Applying The Universal Scalability Law to Distributed Systems by Dr. Neil J. Gunther. DSConf 2019, March 16, 2019. `https://www.youtube.com/watch?v=cuieNj3NrZ8`.
5. N. J. Gunther. *Guerrilla Capacity Planning*. Springer, 2007.
6. Apache Software Foundation. ZooKeeper Administrator's Guide. ZooKeeper 3.9.3 documentation. `https://zookeeper.apache.org/doc/r3.9.3/zookeeperAdmin.html`.
7. MIT 6.824: Distributed Systems. ZooKeeper. Lecture 8, Spring 2020. `https://www.youtube.com/watch?v=pbmyrNjzdDk`.
8. R. Jain. *The Art of Computer Systems Performance Analysis*. Wiley, 1991.
9. B. Gregg. *BPF Performance Tools*. Addison-Wesley, 2019.
10. B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears. Benchmarking cloud serving systems with YCSB. In *ACM Symposium on Cloud Computing (SoCC)*, 2010.
11. A. Ledenev. Pumba: chaos testing and network emulation for Docker containers. `https://github.com/alexei-led/pumba`.

A Stage I experiments not repeated in Stage II

These runs were made on the Stage I laptop with the servers in Docker containers, one run per cell and no confidence intervals. They come from a different testbed and are kept only as context.

YCSB. We ran YCSB [10] with its ZooKeeper binding on 3 servers (Table 10). At 16 threads the read-heavy workloads B and C reach about 6.7 times the throughput of the 50/50 workload A, and in A an update takes 3–4.5 times as long as a read on average (3,713 against 1,078 μs). B and C peak at 16 threads. Workload A jumps from 1,723 to 5,964 ops/s between 16 and 32 threads; with a single run per cell we cannot tell whether this is a real effect or noise, warm-up or garbage collection.

Leader failover. During a continuous benchmark on 3 servers we found the leader with `stat` and killed its container. The two survivors kept a majority and continued, and ZooKeeper logged an election of 211 ms (216 and 235 ms in a repetition). This is ZooKeeper's measurement of the election only; we did not measure the outage seen by clients.

Table 10. YCSB throughput (ops/s), Stage I laptop, 3 servers, one run per cell.

Workload	1 thread	4	16	32
A (50/50 read/update)	790	1316	1723	5964
B (95/5)	3544	6927	11436	8080
C (100% read)	4538	9074	11913	8900

Network delay. We added 20±2 ms to the egress of every server container with Pumba [11] (which drives `tc netem` inside each container's network namespace) and repeated the sweep with the quiet client (Table 11). Throughput per client stays at about 21.5 ops/s, one operation per 46.5 ms, at every load: the clients wait on the delay and the servers never saturate within 80 clients. The median read took 22.7 ms (one delayed hop) and the median write 106 ms (about five: forward, proposal, acknowledgment, commit, reply), as Fig. 1 predicts for a client attached to a follower. We did not fit the USL to these data.

Table 11. Throughput (ops/s) with and without a 20 ms egress delay, Stage I laptop, 3 servers, one run per cell.

Clients	1	5	10	20	40	80
No delay	2386	7016	11680	14832	16238	16255
20 ms delay	20.6	108.9	216.7	434.6	865.5	1709

B Per-stage times at every load level

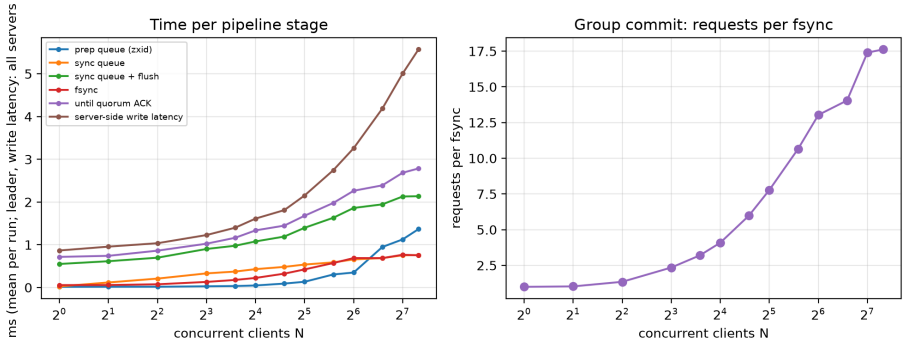


Fig. 4. Leader per-stage times (left) and requests per `fsync` (right) at every level of the USL curve.

C Model checks

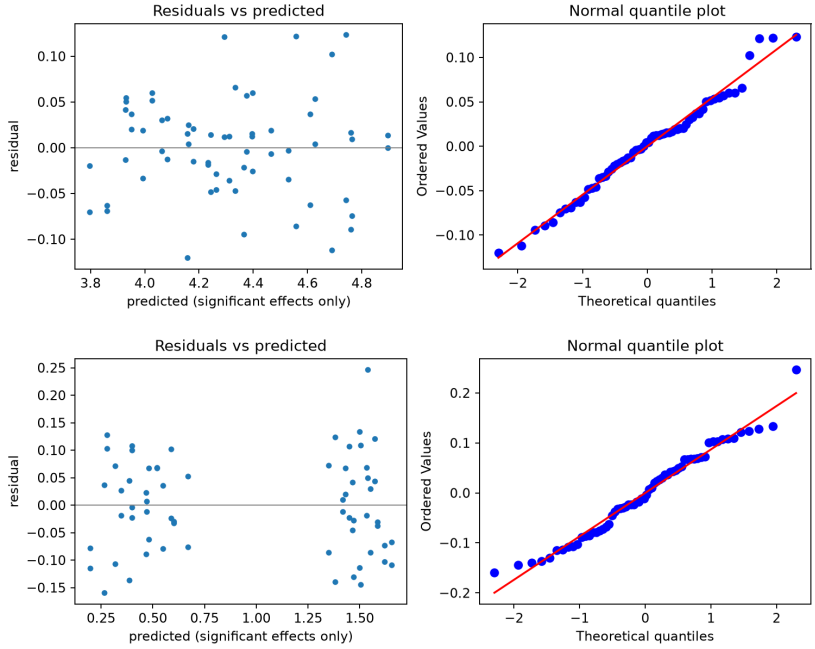


Fig. 5. Residuals of the $\log_{10}$ effects model against the predicted value (left) and normal quantile plots (right): throughput (top) and write p99 (bottom).